

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 - PSEUDOCODE WRITING TASK

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO5 - Naturalization (BL4, BL6)	Assessment:	Assessment Tool 1 - Pseudocode Writing Task
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	K. Ragavi	Reg. No.:	192524364
Section / Batch:	B.Tech. AI & DS	Date:	28/08/2026

Code of Conduct

I K. Ragavi (192524364) (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: K. Ragavi

Instructions

- This test contains 5 Pseudocode Writing Task questions. Total: 100 Marks.
- When a student answer using pseudocode writing task should evaluate based on the ability to design clear, logical, and efficient pseudocode solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

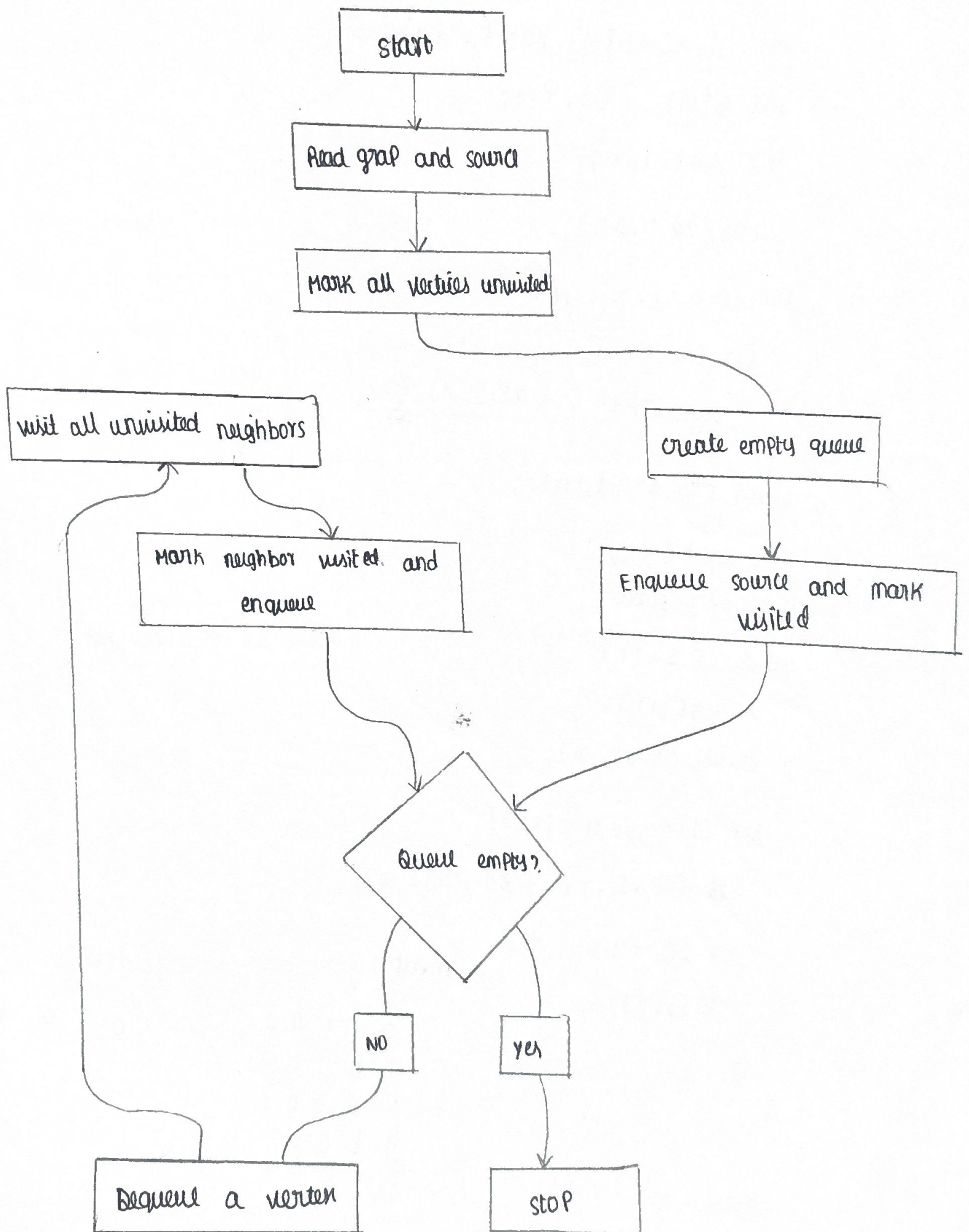
Section / Topic	Focus	Q Nos	Marks
Minimum Spanning Tree	Kruskal's Algorithm	1	20
Graph Sorting	Topological Sort	2	20
Shortest Path Algorithm	MST & Dijkstra's Algorithm	3	20
Shortest Path Algorithm	Dijkstra's Algorithm	4	20
Minimum Spanning Tree	Prim's or Kruskal's Algorithm	5	20
GRAND TOTAL:			100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. convert the flowchart for Breadth first search traversal into code.




```
#include <stdio.h>
```

```
int main() { stdio.h }
```

```
int n, a[10][10], v[10] = {0};
```

```
int q[10], f = 0, r = 1;
```

```
int start, i, x;
```

```
scanf("%d", &n);
```

```
for (i = 0; i < n; i++)
```

```
    for (x = 0; x < n; x++)
```

```
        scanf("%d", &a[i][x]);
```

```
scanf("%d", &start);
```

```
v[start] = 1;
```

```
q[++r] = start;
```

```
while (f <= r) {
```

```
    x = q[f++];
```

```
    printf("%d", x);
```

```
    for (i = 0; i < n; i++) {
```

```
        if (a[x][i] == 1 && v[i] == 0) {
```

```
            v[i] = 1;
```

```
            q[++r] = i;
```

```
        }
```

```
    }
```

```
}
```

```
return 0;
```

```
}
```

Input :

5

0	1	1	0	0
1	0	0	1	0
1	0	0	0	1
0	1	0	0	1
0	1	0	0	1
0	0	1	1	0
0				

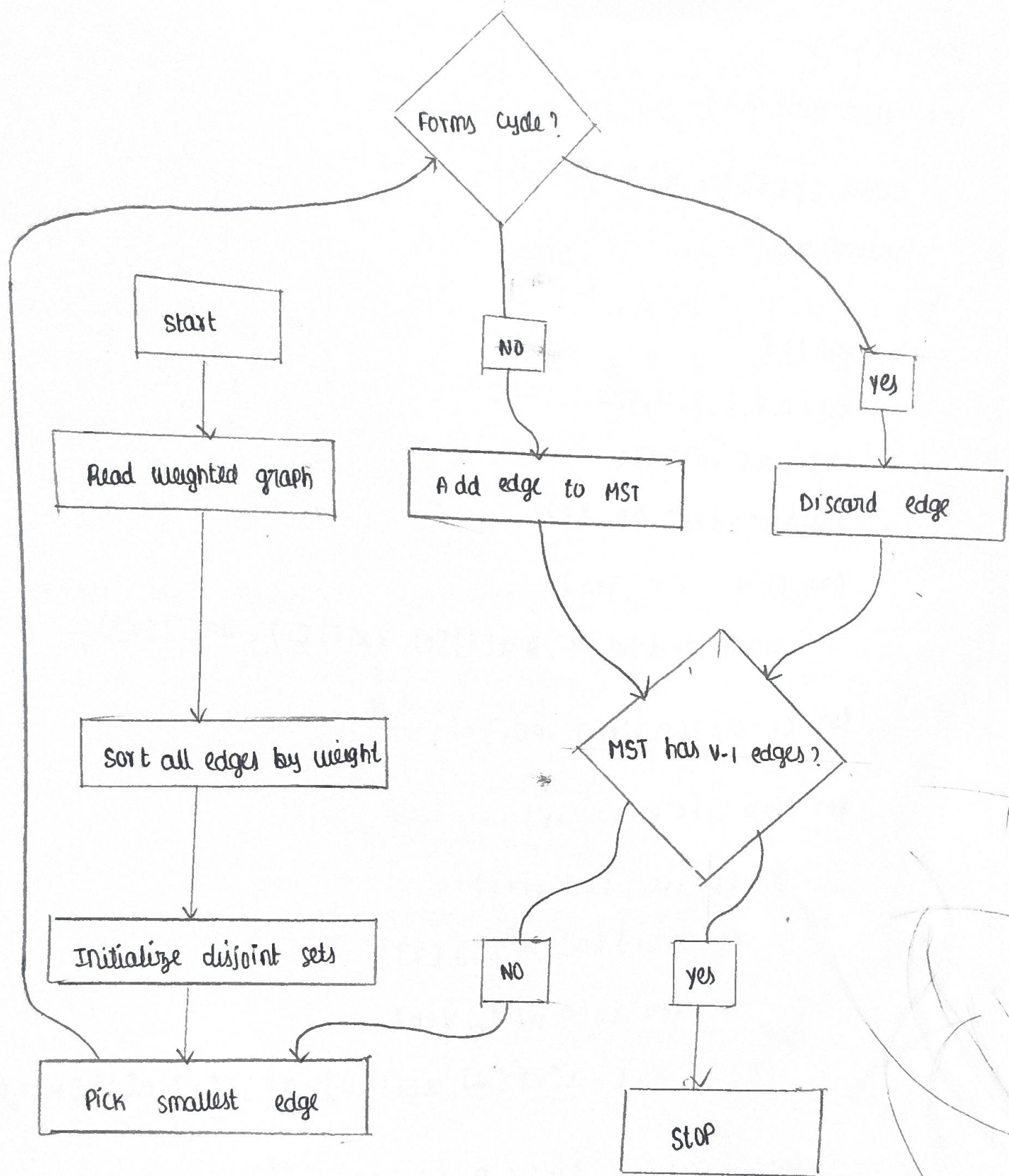
Output :

0 1 2 3 4

Time complexity: $O(V^2)$

Space complexity: $O(V)$

2. convert the flowchart for Kruskal's algorithm into code.




```
#include <stdio.h>
```

```
int P[10];
```

```
int find (int x) {
```

```
    while (P[x] != x) x = P[x];
```

```
    return x;
```

```
}
```

```
int main () {
```

```
    int n, e, i, j, u, v, w, t, c = 0;
```

```
    int a[20][3];
```

```
    scanf ("%d %d", &n, &e);
```

```
    for (i = 0; i < e; i++)
```

```
        scanf ("%d %d %d", &a[i][0], &a[i][1], &a[i][2]);
```

```
    for (i = 0; i < n; i++) P[i] = i;
```

```
    for (i = 0; i < e - 1; i++)
```

```
        for (j = i + 1; j < e; j++)
```

```
            if (a[i][2] > a[j][2])
```

```
                for (v = 0; v < 3; v++)
```

```
                    t = a[i][v], a[i][v] = a[j][v], a[j][v] = t;
```

```
    for (i = 0; i < e && c < n - 1; i++) {
```

```
        u = find (a[i][0]);
```

```
        v = find (a[i][1]);
```

```
        if (u != v) {
```

```
            printf ("%d %d : %d\n", a[i][0], a[i][1], a[i][2]);
```

```
            P[u] = v;
```


c++ ;

}

}

}

Input:

4	5	
0	1	10
0	2	6
0	3	5
1	3	15
2	3	4

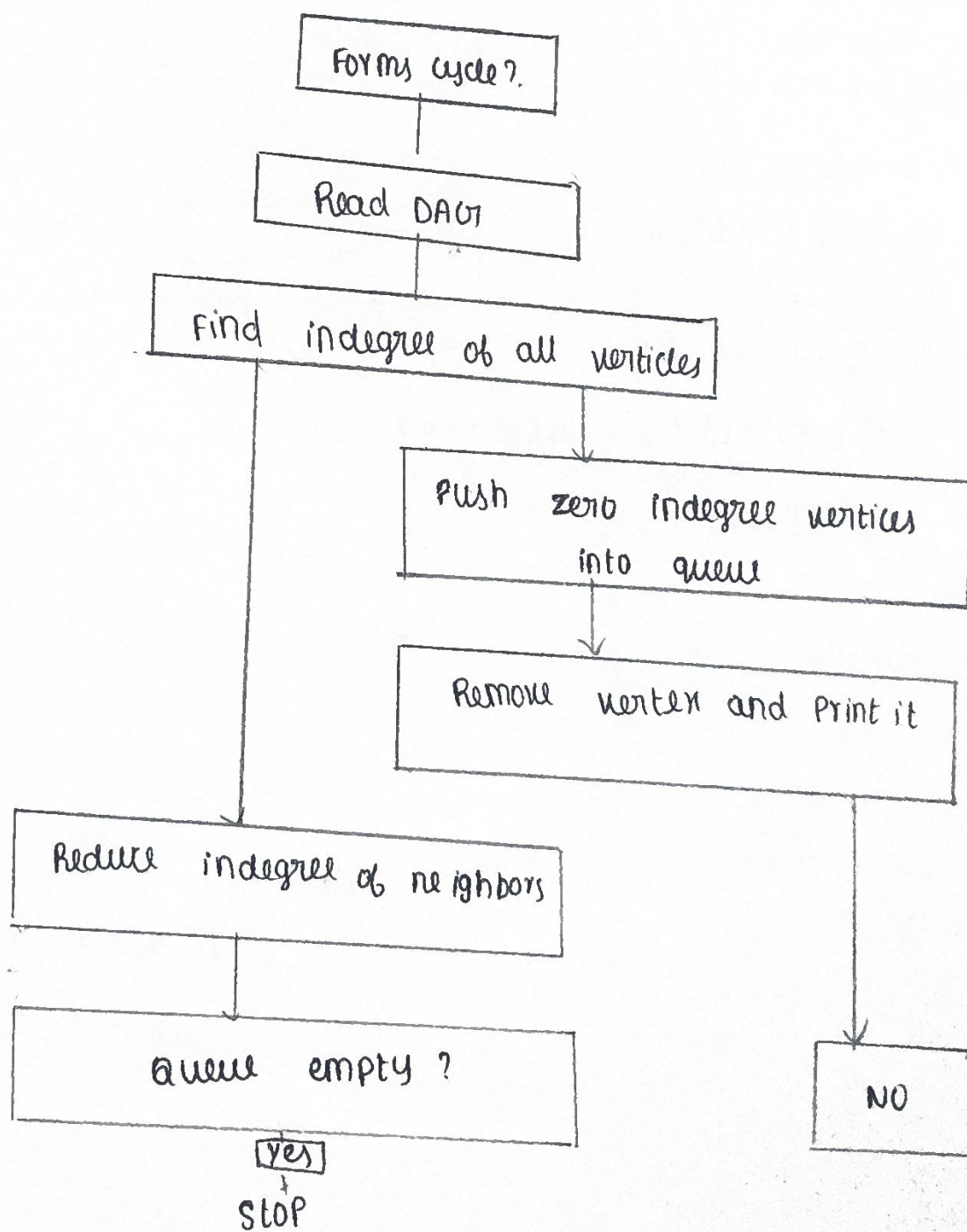
Output :

2-3 : 4

0-3 : 5

0-1 : 10

3. convert the flowchart for topological sort into code.



```
#include <stdio.h>
```

```
int main() {
```

```
int n, a[10][10], in[10] = {0}, q[10], f=0, r=-1, i, j, v;
```

```
scanf ("%d", &n);
```

```
for (i=0; i<n; i++)
```

```
for (j=0; j<n; j++) {
```

```
scanf ("%d", &a[i][j]);
```

```
if (a[i][j]) in[j]++;
```

```
}
```

```
for (i=0; i<n; i++)
```

```
if (in[i] == 0 q[f+r]=i;
```

```
while (f <= r) {
```

```
v = q[f++];
```

```
printf ("%d", v);
```

```
for (i=0; i<n; i++)
```

```
if (a[v][i] && --in[i] == 0)
```

```
q[f+r]=i;
```

```
}
```

```
return 0;
```

```
}
```

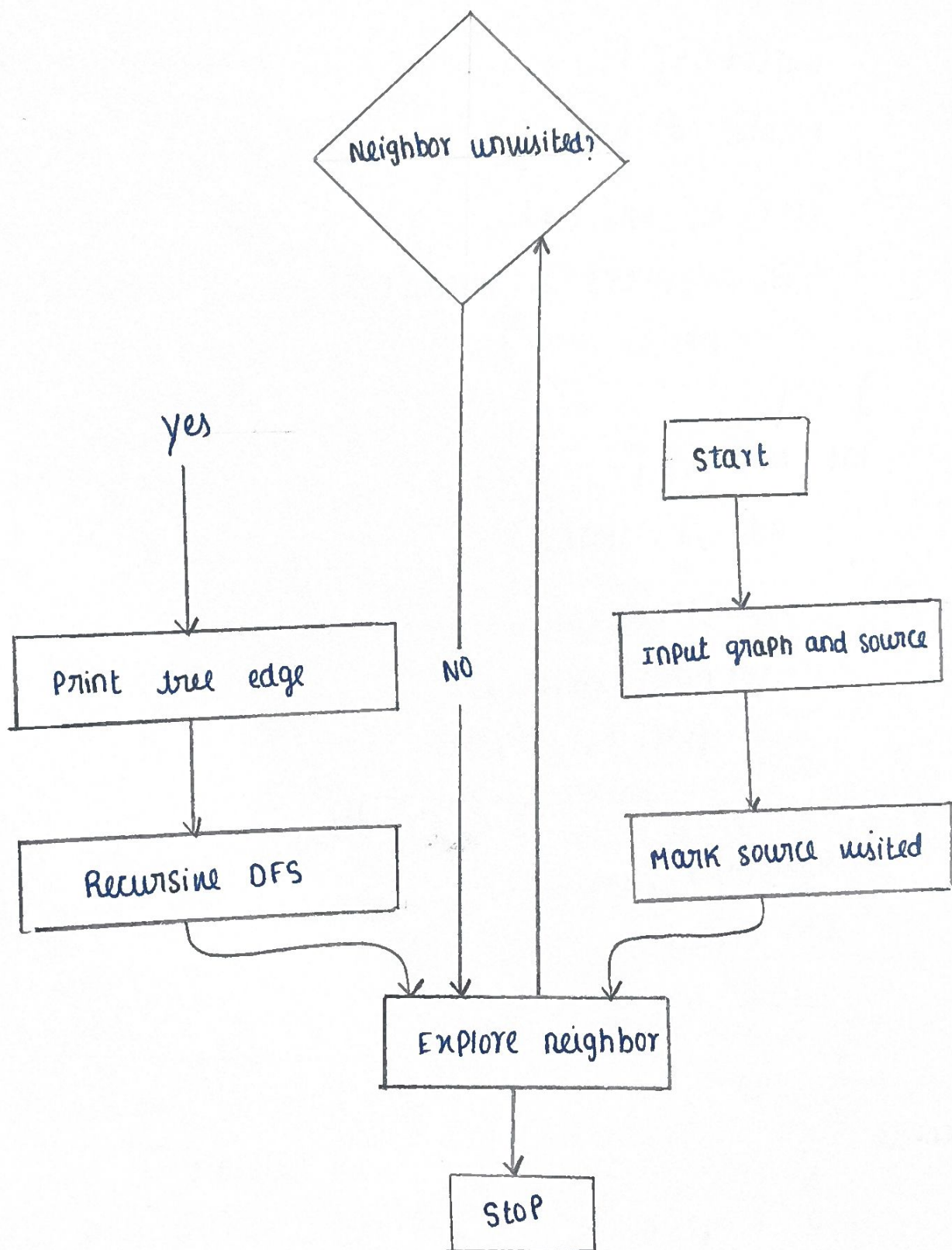
Input:

6	0			
0	1	1	0	0
0	0	0	0	0
0	0	0	1	0
0	0	0	0	1
0	0	0	0	1
0	0	0	0	0

Output :

0	1	2	3	4	5
---	---	---	---	---	---

4. convert the flowchart for generating a DFS tree into code




```
#include <stdio.h>
```

```
int a[10][10], visited[10].n;
```

```
void DFS (int v) {
```

```
    int i;
```

```
    visited[v] = 1;
```

```
    printf("%d", v);
```

```
    for (i=0; i < n; i++)
```

```
        if (a[v][i] && !visited[i])
```

```
            DFS(i);
```

```
}
```

```
int main () {
```

```
    int i, j, start;
```

```
    scanf("%d", &n);
```

```
    for (i=0; i < n; i++)
```

```
        for (j=0; j < n; j++)
```

```
            scanf("%d", &a[i][j]);
```

```
    scanf("%d", &start);
```

```
    DFS (start);
```

```
    return 0;
```

```
}
```

Input :

```
5
0 1 1 0 0
1 0 0 1 0
1 0 0 0 1
0 1 0 0 1
0 0 1 1 0
0
```

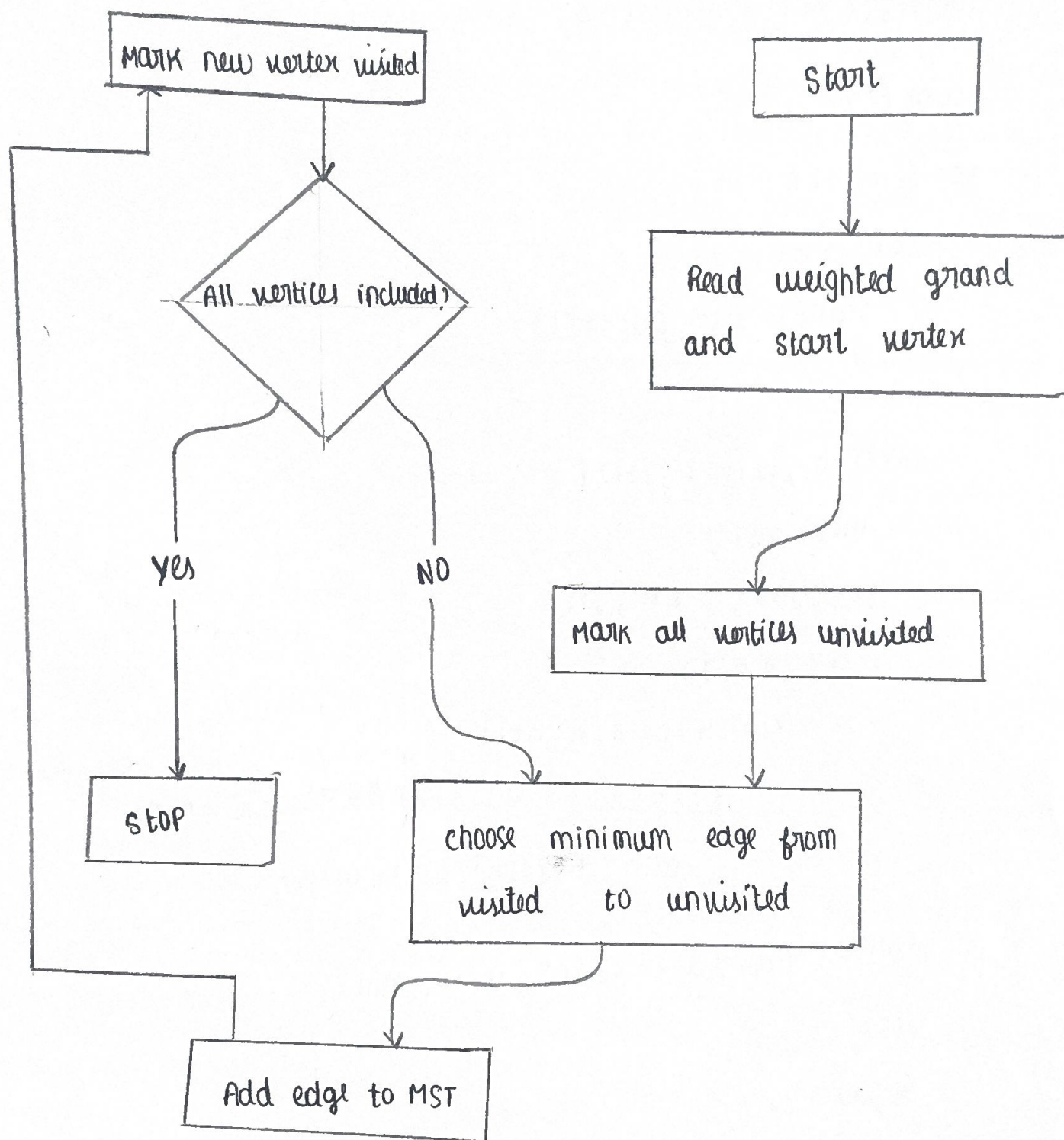
Output :

```
0 1 3 4 2
```

Time complexity : $O(v^2)$

Space complexity : $O(v)$

5. convert the flowchart for Prim's algorithm into code.




```
#include <stdio.h>
```

```
int main() {
```

```
    int n, a[10][10], v[10] = {0};
```

```
    int i, j, k, min, u, w, cost = 0;
```

```
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++)
```

```
        for (j = 0; j < n; j++)
```

```
            scanf("%d", &a[i][j]);
```

```
    v[0] = 1;
```

```
    for (i = 0; i < n - 1; i++) {
```

```
        min = 999;
```

```
        for (j = 0; j < n; j++)
```

```
            if (v[j])
```

```
                for (k = 0; k < n; k++)
```

```
                    if (!v[k] && a[j][k] && a[j][k] < min)
```

```
                        min = a[j][k], u = j, w = k;
```

```
        printf("%d - %d : %d\n", u, w, min);
```

```
        cost += min;
```

```
        v[w] = 1;
```

```
    }
```

```
    printf("minimum cost = %d", cost);
```

```
}
```

Input :

5				
0	2	0	6	0
2	0	3	8	5
0	3	0	0	7
6	8	0	0	9
0	5	7	9	0

Output :

0-1 : 2

1-2 : 3

1-4 : 5

0-3 : 6

minimum cost = 16